

Lab 3: Component Modelling & Architectural Pattern Selection

Architecture Selection

We chose Microservices Architecture for the Food Bank Surplus Redistribution Platform.

The platform is decomposed into six components: Client App, API Gateway, User & Verification Service, Food Listing Service, Reservation Service and Notification Service. They interact through seven interfaces (IGatewayAPI, IIdentity, IFoodListing, IReservation, IBatchAvailability, INotify and IShelterDirectory) wired by eight assembly connectors, and each backend service owns its private datastore.

Two Reasons for This Choice

- **1. Bursty, uneven load on different features:** Restaurants and grocery stores post surplus mostly around closing and meal times, and every high-quantity listing must trigger instant push alerts to all shelters within 5 km. The Notification and Listing services therefore face far heavier peaks than account management. Microservices let us scale only these hot services, instead of scaling the whole application as a Layered or single-server design would.
- **2. Food expiry demands high availability and fault isolation:** A perishable batch is only useful before its expiry timer ends, so donors must always be able to post and shelters to reserve. If the push-notification provider slows down or fails, the Listing and Reservation services keep working. A Client-Server design would make one central server a single point of failure, and independent deployment lets us change one service (e.g. dietary tags) without redeploying or risking the rest.

Security Advantage

All traffic enters through the **API Gateway**, which checks each request's token with the User & Verification Service (IIdentity), so only verified shelters can reserve food and only registered donors can post it. Because every service owns a private datastore and exposes only narrow interfaces, a compromise of one service (e.g. Notification) does not expose donor, shelter or reservation data; all links use HTTPS/TLS.

Performance Benefit

Independent horizontal scaling plus asynchronous messaging: when a donor posts a batch, the Listing Service publishes an INotify event and returns immediately, while the Notification Service fans out alerts in parallel to nearby shelters. This keeps posting fast under peak load, and the Reservation Service locks a batch through IBatchAvailability so two shelters cannot claim the same food.

Why Not the Other Styles?

Layered: the whole application is deployed and scaled as one unit, with layer-to-layer overhead on every request. **Client-Server:** a centralised server is a single point of failure and a scalability bottleneck during alert spikes. **Trade-off accepted:** microservices add operational complexity and network latency; we reduce this with a single API Gateway entry point and asynchronous events for non-critical work.